



Graph Representation and Chromatic Computation for Engineering Applications

R. Manimegalai¹(✉), J. Anitha²(✉), J. Dharineesh¹,
and M. Chandramouleeswaran³

¹ PSG Institute of Technology and Applied Research,
Coimbatore 641004, Tamil Nadu, India
drmm@psgitech.ac.in

² Rajalakshmi Institute of Technology, Chennai 600124, Tamil Nadu, India
anithaharish78@gmail.com

³ Hindustan Institute of Technology and Science, Chennai 603107, Tamil Nadu, India

Abstract. The chromatic number of a graph G , $\chi(G)$ represents the minimum number of colors required to color the vertices of a graph such that no adjacent vertices share the same color. This paper provides a focused exploration of the chromatic number, emphasizing its mathematical significance and practical applications. In this work, the behavior of the chromatic number across various graph structures is investigated. Leveraging the computational tools, we develop algorithms to analyze the chromatic number across diverse graph structures and explore its implications for graph coloring problems. Additionally, we introduce visualization techniques to graphically represent graph coloring and thereby provide intuitive insights into the dynamics of the chromatic number. Experimental results include the visual representation of the graph, chromatic number of the given graph, and the execution time of the algorithm for various types of graphs. The algorithm can handle any number of nodes and the results are produced for the appropriate graph type for varying number nodes. This paper discusses the engineering applications of chromatic number in several fields such as education, networking, circuit design, integrated circuits, and so on.

Keywords: Chromatic Number · Graph Coloring · Networks · Scheduling · Register Allocation · Map Coloring and Cartography · Routing and Wavelength Assignment

1 Introduction

The chromatic number of the graph is defined as the minimum number of colors needed to color the vertices such that adjacent vertices should be in different colors. The chromatic number emerges as a fundamental property with profound implications across diverse applications. For a graph $G = (V, E)$, where V represents a finite non-empty set of vertices and represents a set of edges

consisting of unordered pairs of different vertices, the chromatic number $\chi(G)$ denotes the minimum number of colors required to color each vertex of G . The constraint is that adjacent vertices should be in different colors. This concept lies at the core of graph coloring problems, which are of paramount importance in both theoretical and practical contexts. This work aims to contribute to the theoretical understanding of graph coloring problems and development of efficient graph coloring algorithms. Chromatic numbers can be implemented for all types of graphs, due to this diverse nature it can be used in multiple applications. In this work, we have also created an algorithm to compute chromatic numbers for various types of graph and varying node size. Basic definitions and terminologies related to chromatic number and graph coloring are given below [1, 19, 20]. A **complete graph** is an undirected graph in which every vertex is connected with every other vertex by a unique edge. It is denoted by K_n , where n represents the vertex count. The total edges in a complete graph are given by the formula $|E| = \frac{n(n-1)}{2}$. A **cycle graph** forms a single cycle, meaning it is a closed loop with no self-intersections. It is denoted by C_n , where $n \geq 3$ represents the vertex count. The total edges in a cycle graph are $|E| = n$.

A **star graph** consists of a single central vertex which is connected to all other vertices. It is denoted by $K_{1,n}$, where n represents the outer vertex count. The total edges in a star graph are $|E| = n$.

A **fan graph** consists of a single central vertex which is connected to all other vertices arranged in a sequential manner. It is denoted by F_n , where n represents the outer vertex count. The total number of edges in a fan graph are $|E| = 2n - 2$.

A **path graph** consists of vertices which are arranged in a linear sequence, and each vertex is adjacent to its immediate neighbors. It is denoted by P_n , where n represents the vertex count. The total number of edges in a path graph are $|E| = n - 1$.

A **binary tree** represents a binary tree, where each vertex has at most two children. It is denoted by T_n , where n represents the number of vertices in the tree. The total number of edges in a binary tree are $|E| = n - 1$.

A **Sibling tree** represents a tree structure where each vertex, except the root, has exactly one parent and one or more siblings. It is denoted by ST_n^1 , where n represents the number of vertices in the tree. The total number of edges in a Sibling tree are $|E| = n - 1$.

The **chromatic number**, $\chi(G)$, of a graph G is the smallest number of colors for $V(G)$ so that adjacent vertices are colored differently. The formula for the Chromatic Number is $\chi(G) = \min \{k : \text{there exists a proper } k\text{-coloring of } G\}$.

1.1 Role of Chromatic Number Computation in Engineering Application

Examination Scheduling: In scheduling, the well-known problem is examination scheduling [2] in which no person should participate in two or more exams simultaneously. The aim is to plan the exams so that the total periods required for the examinations are minimized. The primary purpose is to allocate time slots to

each exam so that the total number of periods required for all examinations is minimized. Reducing the duration of exams that are spread, makes the process more efficient. The problem is complicated by the availability of rooms and the number of students enrolled in each course. The problem can be solved with different strategies such as heuristic algorithms, and graph coloring which are often employed. These methods help to minimize the time slots while ensuring that no student was overlapping exams. Table 1 represents the courses and their corresponding colors. In Fig. 1(a), the nodes represent the courses and nodes are colored in gray which represents the courses before allocation of time for conducting the examination. In Fig. 1(b), the nodes are colored using the Chromatic number algorithm which represents the courses after allocation of time for conducting examination.

Table 1. Courses and their Corresponding Colors

courses	Color
C4, C5, C6	Blue
C2, C1	Green
C3	Red

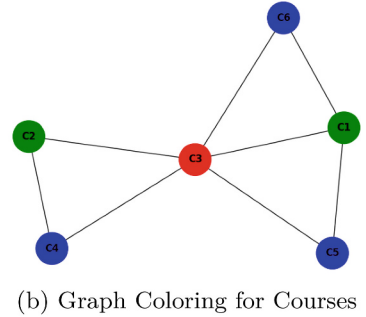
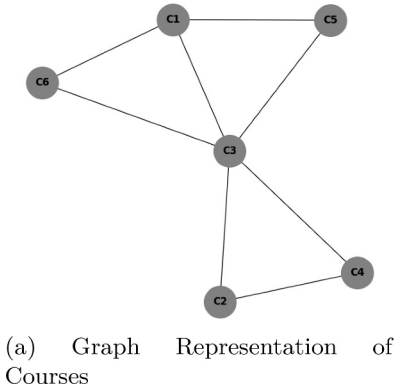


Fig. 1. Examination Scheduling using Chromatic Number.

Routing and Wavelength Assignment (RWA): It is an important issue in designing routing protocols. In an optical network, data is transmitted in the form of waves. Signals with different wavelengths can be transmitted over the same link using a technology called Wavelength Division Multiplexing (WDM) [3]. If two signals are transmitted through the same link, the signals interfere with each other which leads to error in the transmitted data. For this application, the network is represented as a graph, where the vertices are network nodes, and the edges are the physical connection. The chromatic number of this

graph represents the minimum number of distinct colors needed to assign to the edges incident to each node such that no two links having the same color transmit the signal with the same wavelength [3]. In Fig. 2, Blue color nodes represent the access points where data is either injected into or extracted from the optical network. The orange nodes represent the optical switch that manages the routing and the edges represent the light path. The two light paths of the same wavelength take different paths which represent the chromatic number of the graph.

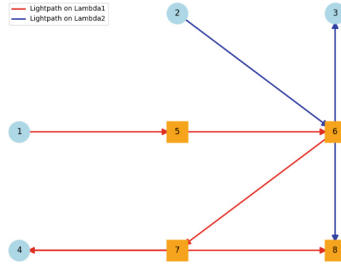


Fig. 2. Lightpath in Optical Networks.

Map Coloring: It is a classic problem in graph theory in which no two adjacent regions have the same color. The minimum number of colors used is the chromatic number of that map. Map coloring can be implemented in study and practice of making and using maps called cartography. Map coloring can be solved using various methods but the most used methods are backtracking and chromatic number. Figure 3 shows that the World map is colored in green, yellow, blue and red to illustrate the four color theorem. The four color theorem states that only four colors are needed to color any type of map such that no two adjacent regions are in the same color [4]. This map considered only land boundaries [21].

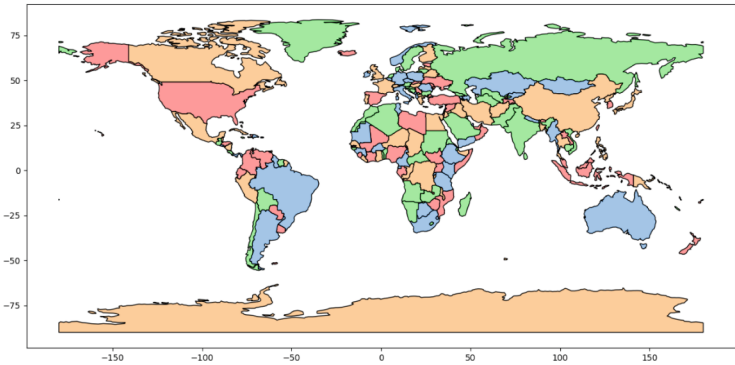


Fig. 3. Map Coloring on World Map using Four Color Theorem.

Register Allocation: Each node represents a register, and an edge is drawn between the nodes. During the register allocation phase of the compiler design, the compiler assigns intermediate values to storage locations in the process [5]. One approach is to allocate variables within a loop as a coloring problem in a circular arc graph [6]. Strong techniques are proposed for global and local allocation problems but graph coloring covers the critical aspects of global allocation. One way to solve global register allocation is by using priority-based coloring algorithm [7]. The goal is to assign a register to each variable such that no two interfering variables are assigned to the same register. The minimum colors represent the minimum number of registers required for the program. Figure 4(a) shows the register allocation graph before assigning the variables. Figure 4(b) shows that the register allocation graph is used in compiler optimization, specifically for allocating variables to a limited number of CPU registers. The adjacent registers cannot have the same variable. Nodes with the same color represent variables that can be allocated to the same register. Here the chromatic number algorithm is used to minimize the number of colors used and also reduce the number of registers required for doing computation.

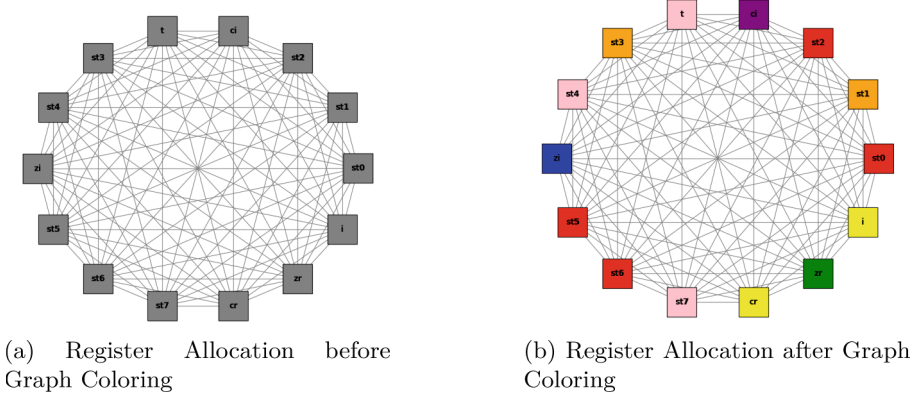


Fig. 4. Register Allocation using Chromatic Number.

1.2 Literature Review

Discrete optimization is one of the optimization techniques in which the variables can take only discrete values, typically integers or other countable sets. E. Malaguti et al. have proposed various algorithms to provide this optimization technique. They have proposed a heuristic algorithm that improves the performance. For the initial feasible solution and with a suitable set of models MMT algorithm is used. The Tabu Search algorithm is used to determine the maximal stable graph G , with weight concerning vector π . Grosso et al. have used effective neighborhoods for local search algorithms to design the algorithm for the

maximum clique problem [8]. The swap move involves removing a vertex from the current clique and one or more new vertices to join the clique. The Tabu search algorithm is also used to find the chromatic number of a random graph [9]. The work in [10] has proposed the benchmark system which includes random graph vertices that are randomly distributed within a unit square according to the DIMACS benchmark.

Bart M.P. Jansen and Jesper Nederlof have proposed a methodology to find the chromatic number through graph decomposition using matrix rank [11]. The proposed algorithms are deterministic algorithm with runtime of $O(2\omega \cdot ct\omega)$ and randomized algorithm with runtime of $O(2 \cdot ct\omega)$ respectively. The complexity of the graph coloring problem is governed by the characteristics of the input which is under the framework of parameterized algorithms. The framework develop algorithms with runtime of $f(k) \cdot n^{O(1)}$, where k measures the complexity of the instance. The work in [11] discusses how to show graph decompositions using small edge separators that can be used to solve chromatic coloring. Alon et al. have proposed a way to enlarge the chromatic number of a random graph [12]. Akhan Akbulut and Güray Yılmaz have proposed a solution for the exam scheduling problem which all universities face in each semester [13]. They consider the final exam weeks of the universities as the main problem which can solve the exam scheduling problem. The idea is that different exams are scheduled in different halls at the same time. The goal is to use the complete hall capacity and decrease cheating in the exams. Students writing different exams can sit closer so that the complete hall is used and also cheating is reduced. Carter and Laporte have proposed exam scheduling problems for four classes, namely, constraint satisfaction approaches, generalization search technique, sequential methods, and classification method [14]. Exams are represented using graphs. Each exam is considered as a node and the edges represent the conflict among the exams. Period of a day represents a color so that no two adjacent nodes can have the same color. Ganguli et al. state that the timetable scheduling problem is NP-Hard. Thus a heuristic approach is used to find the nearest optimal solution in a reasonable time [15].

Matousek et al. have studied topological lower bounds for the chromatic number of a graph. For planar graphs, determining 3-colorability is algorithmically difficult, and beyond planar graphs, the gaps between the upper and the lower bounds that one can reasonably obtain for $\chi(G)$, may be huge. The most famous coloring problem is the four-color problem, which questions whether every planar graph can be colored with only four colors [16]. Daniel Sanders et al. state that the four-color theorem strongly matches that every loopless planar graph admits a 4-coloring [17]. Integral graphs are a combination of circular graphs and integral graphs with maximum vertices of $n - 1$. This type of graph is used in interconnection networks in parallel and distributed computing and so on. Aleksandar et al. investigated recent research focused on the chromatic number of the integral circulant graph, Klotz and Sander that suggested the chromatic number divide the order of the graph [18].

2 Implementation of Chromatic Number Algorithm

In this work, multiple algorithms are implemented to generate various graph structures and compute their chromatic numbers. Empirical analysis is done to uncover the structural intricacies that influence the chromatic number across different types of graphs. The algorithm is separated into modules and grouped to form a package. Vertex class, Edge class, a class for various graph structures, and a main class for getting the number of nodes and type of graph from the user as input, are the modules of the package. The modularity of the application allows for easy maintenance, customization and extension.

The proposed algorithm starts by building a given graph structure and assigning a color to each node in which the adjacent nodes do not share the same color. It then iterates through each of the vertices and finds the smallest available color that is not used by its neighbors by keeping track of the coloring dictionary. After processing, it determines the chromatic number of the graph. The result not only computes chromatic number accurately but also ensures faster execution times, making it useful for practical applications such as network design, exam scheduling, resource allocation, and map coloring.

The execution time of the algorithm is calculated by subtracting the end time and start time of the algorithm. Visualization of Chromatic number computation is also presented in the paper thereby providing intuitive insights into the chromatic number's behavior and its correlation with graph structure. Through visualization, users can easily observe and understand the relationship between graph structure and chromatic number. The visualization of the graph is produced by a graphics module of the algorithm.

The algorithm starts by initializing a dictionary to map vertices to their positions and another to store the color assigned to each vertex. For each vertex, it initializes a set of available colors and removes colors already used by its adjacent vertices. If no colors are available, it prints an error indicating that the graph cannot be properly colored. Otherwise, it assigns the smallest available color to the vertex. After coloring all vertices, it maps colors to unique values, colors the nodes, and displays the graph with the colored nodes. The maximum color used determines the chromatic number. The algorithm also measures and returns the execution time, which is the difference between the start and end times of the process. If no valid coloring is found, it returns zero as the chromatic number along with the execution time. The generalized algorithm is tabulated as Table 2 for diverse graph types.

Algorithm 1. ChromaticNumber()

```

1: start_time  $\leftarrow$  Current time
2: graph_coloring  $\leftarrow$  Empty dictionary
3: for each vertex in vertices do
4:   available_colors  $\leftarrow$  Set of integers from 1 to the number of vertices
5:   for each edge in vertex.edge_list do
6:     neighbor_color  $\leftarrow$  graph_coloring.get(edge.destination_vertex_id)
7:     if neighbor_color is not None then
8:       Remove neighbor_color from available_colors
9:     end if
10:  end for
11:  if available_colors is empty then
12:    print "Graph is not properly colored."
13:  else
14:    chosen_color  $\leftarrow$  Minimum value in available_colors
15:    graph_coloring[vertex.state_id]  $\leftarrow$  chosen_color
16:  end if
17: end for
18: end_time  $\leftarrow$  Current time
19: execution_time  $\leftarrow$  end_time  $-$  start_time
20: if graph_coloring is not empty then
21:   chromatic_number  $\leftarrow$  Maximum value in graph_coloring.values()
22:   return chromatic_number, execution_time
23: else
24:   return 0, execution_time
25: end if

```

Table 2. Algorithm for Coloring Various Types of Graph

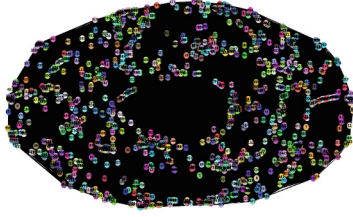
Graph type	Input	Algorithm	Chromatic number
Path Graph	Path graph of length n	1. Name nodes 1, 2, ..., n. 2. Color nodes left to right, adjacent nodes get different colors	$\chi(P_n) = 2$
Cycle Graph	Cycle graph of length n	1. Name vertices 1, 2, ..., n. 2. Color nodes clockwise, adjacent nodes get different colors	$\chi(C_n) = 2$ when n is even and 3 when n is odd
Star Graph	Star graph with n+1 nodes	1. Name vertices 0, 1, 2, ..., n. 2. Color central node, then pendant nodes with different colors	$\chi(k_{1,n}) = 2$
Fan Graph	Fan graph with n+1 nodes	1. Name nodes 1, 2, ..., n. 2. Color the central node, then adjacent nodes with different colors	$\chi(F_n) = 3$
Complete Graph	Complete graph with n nodes	1. Name nodes 1, 2, ..., n. 2. Color each node with a unique color	$\chi(K_n) = n$
Binary Tree	Binary tree with n nodes	1. Name nodes 1, 2, ..., n. 2. Color root, then adjacent nodes with different colors	$\chi(T_n) = 2$
Sibling Tree	Sibling tree with n nodes	1. Name nodes 1, 2, ..., n. 2. Color root, then adjacent nodes with different colors	$\chi(ST_n^1) = 2$

Table 3. Execution Time for Various Networks of n Nodes

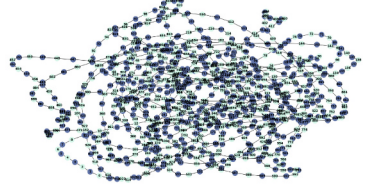
Types of graphs	Execution time for n nodes (sec)				Chromatic number
	128	256	512	1024	
Complete graph K_n	0.14	0.73	5.67	40.72	128, 256, 512, 1024
Cycle graph C_n	0.003	0.007	0.021	0.088	2
Star graph $K_{1,n}$	0.002	0.006	0.018	0.079	2
Fan graph F_n	0.004	0.008	0.035	0.116	3
Path graph P_n	0.003	0.008	0.025	0.086	2
Binary Tree T_n	0.001	0.002	0.004	0.015	2
Sibling tree ST_n^1	0.001	0.002	0.004	0.011	2

3 Experimental Results

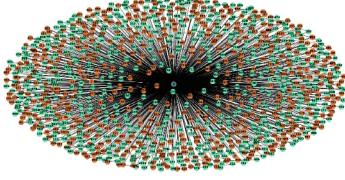
The algorithm is capable of handling an arbitrary number of nodes for various graph types, including complete graph, cycle graph, fan graph, star graph, path graph, binary tree, and sibling tree. We have tabulated the results for 128, 256, 512, and 1024 nodes, detailing the execution time and chromatic number for each graph type. The algorithm also generates visualizations of the graphs, where the chromatic number is represented through distinct graph coloring which helps to easily visualize the output. Our findings indicate that the chromatic number remains constant across all graph types, except for complete graphs, where it equals the number of nodes. Notably, the execution time is significantly higher for complete graphs, as the complexity of determining the chromatic number increases with the number of nodes. But GPU-based processing can reduce the execution time by parallelizing the computation. Table 3 provides a detailed outline of execution time in seconds and chromatic number for n number of nodes across the different graph types. Figure 5 shows the visualization for the graph that we made through our algorithm for 128, 256, 512, and 1024 nodes. The chromatic number has extensive applications in real-world scenarios in areas such as scheduling, register allocation in compilers, map coloring, and routing. These use cases underscore the importance of efficient algorithms for determining the chromatic number, especially in complex graph structures.



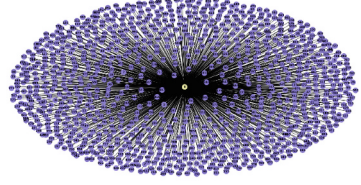
(a) Complete graph $k_{1024}k_{1024}$



(b) Cycle graph C_{1024}



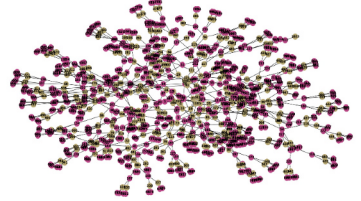
(c) Star graph $K_{1,1024}$



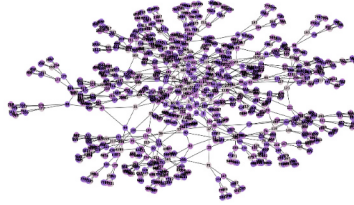
(d) Fan graph F_{1024}



(e) Path graph P_{1024}



(f) Binary Tree T_{1024}



(g) Sibling tree ST_{1024}

Fig. 5. Chromatic Coloring for Different Graph Structures.

4 Conclusions

This paper presents the implementation of algorithms for computing the chromatic number for the complete graph, cycle graph, path graph, fan graph, star graph, sibling tree, and binary tree. The algorithms for different graph types are presented, while the process for finding the chromatic number is outlined in a separate block. The execution time is measured for computing the chromatic number for various graph structures. In addition to numerical data, visualization

for various graph structures are generated, in which the chromatic number is represented through distinct graph coloring. It is observed that the proposed implementation for the chromatic number computation can handle up to 5000 nodes. The results such as execution time in seconds, chromatic number of the respective graph type were tabulated along with neat diagrammatic representation of the graph produced by the algorithm. Applying the proposed chromatic number algorithms to solve the engineering problems such as examination scheduling, max-clique-problem, is an interesting research issue. Use cases of the problem on various applications such as map coloring, register allocation problem in compiler design, and exam scheduling problem in universities are discussed in the paper and can be implemented using the proposed methodology in future.

References

1. Chartrand, G., Zhang, P.: Introduction to graphs. In: Chromatic Graph Theory, pp. 27–52 (2019). <https://doi.org/10.1201/9780429438868-2>
2. Leighton, F.T.: A graph coloring algorithm for large scheduling problems. *J. Res. Natl. Bureau Stand.* **84**, 489 (1979). <https://doi.org/10.6028/jres.084.024>
3. Sharafat, A.R., Ma'rouzi, O.R.: The most congested cutset: deriving a tight lower bound for the chromatic number in the RWA problem. *IEEE Commun. Lett.* **8**, 473–475 (2004). <https://doi.org/10.1109/lcomm.2004.832765>
4. Franklin, P.: The four color problem. *Am. J. Math.* **44**, 225 (1922). <https://doi.org/10.2307/2370527>
5. Cooper, K.D., Torczon, L., Cooper, K.D., Torczon, L.: Register allocation. In: *Engineering a Compiler (Third Edition)*, vol. 12, pp. 663–712 (2023)
6. de Werra, D., Eisenbeis, C., Lelait, S., Marmol, B.: On a graph-theoretical model for cyclic register allocation. *Disc. Appl. Math.* **93**(2–3), 191–203 (1999)
7. Woo, T.-K., Su, S.Y.W., Newman-Wolfe, R.: Resource allocation in a dynamically partitionable bus network using a graph coloring algorithm. *IEEE Trans. Commun.* **39**, 1794–1801 (1991). <https://doi.org/10.1109/26.120165>
8. Grosso, A., Locatelli, M., Pullan, W.: Simple ingredients leading to very efficient heuristics for the maximum clique problem. *J. Heur.* **14**, 587–612 (2007). <https://doi.org/10.1007/s10732-007-9055-x>
9. Hertz, A., de Werra, D.: Using tabu search techniques for graph coloring. *Computing* **39**(4), 345–351 (1987). <https://doi.org/10.1007/bf02239976>
10. Malaguti, E., Monaci, M., Toth, P.: A metaheuristic approach for the vertex coloring problem. *INFORMS J. Comput.* **20**, 302–316 (2008). <https://doi.org/10.1287/ijoc.1070.0245>
11. Jansen, B.M., Nederlof, J.: Computing the chromatic number using graph decompositions via matrix rank. *Theor. Comput. Sci.* **795**, 520–539 (2019)
12. Alon, N., Sudakov, B.: Increasing the chromatic number of a random graph. *J. Comb.* **1**, 345–356 (2010). <https://doi.org/10.4310/joc.2010.v1.n4.a1>
13. Akbulut, A., Yilmaz, G.: University exam scheduling system using graph coloring algorithm and RFID technology. *Int. J. Innov.* **4** (2013)
14. Carter, M.W., Laporte, G.: Recent developments in practical examination timetabling. In: Burke, E., Ross, P. (eds.) *PATAT 1995. LNCS*, vol. 1153, pp. 1–21. Springer, Heidelberg (1996). https://doi.org/10.1007/3-540-61794-9_49

15. Ganguli, I., Sil, J.: Graph clustering based topic modeling using feature learning approach. In: Proceedings of the Workshop Program of the 19th International Conference on Distributed Computing and Networking, pp. 1–2 (2018). <https://doi.org/10.1145/3170521.3170543>
16. Matousek, J., Ziegler, G.M.: Bounds for the Chromatic Number (2002)
17. Robertson, N., Sanders, D., Seymour, P., Thomas, R.: The four-colour theorem. *J. Comb/ Theory Ser. B* **70**, 2–44 (1997). <https://doi.org/10.1006/jctb.1997.1750>
18. Ilić, A., Bašić, M.: On the chromatic number of integral circulant graphs (2010)
19. Combinatorial Optimization (1998)
20. Bar-Noy, A., Motwani, R., Naor, J.: The greedy algorithm is optimal for on-line edge coloring. *Inf. Process. Lett.* **44**, 251–253 (1992). [https://doi.org/10.1016/0020-0190\(92\)90209-e](https://doi.org/10.1016/0020-0190(92)90209-e)
21. White, A.T.: Map-coloring problems. In: North-Holland Mathematics Studies Graphs of Groups on Surfaces - Interactions and Models, pp. 89–106 (2001). [https://doi.org/10.1016/s0304-0208\(01\)80009-8](https://doi.org/10.1016/s0304-0208(01)80009-8)
22. Freimer, R.: Generalized map coloring for use in geographical information systems. In: Proceedings of the 8th ACM International Symposium on Advances in Geographic Information Systems, pp. 167–173 (2000). <https://doi.org/10.1145/355274.355299>